Esercizi Laboratorio 1:

Titolo: "Sistema Ibrido C&C (Command & Control) per Flotta di Droni"

Lo Scenario: Stai sviluppando il software di comunicazione tra una **Stazione di Terra (Server)** e una flotta di **Droni (Client)**. I droni hanno bisogno di due canali di comunicazione separati:

1. **Un canale TCP (Porta 7000):** Usato per il Login, i comandi critici (Decollo, Atterraggio) e le emergenze. Questo canale *deve* rimanere sempre aperto (connesso) finché il drone è in volo.
2. **Un canale UDP (Porta 7001):** Usato per la "Telemetria". Il drone spara continuamente (ogni secondo) le sue coordinate GPS e il livello di batteria. Se un pacchetto UDP si perde nel vento, fa niente, arriverà quello del secondo successivo.

---

1. La Stazione di Terra (SERVER)

Il Server deve gestire contemporaneamente sia connessioni TCP che datagrammi UDP. All'avvio, il Server si mette in ascolto:

- In ***listen()*** sul socket **TCP porta 7000**.
- In ***bind()*** sul socket **UDP porta 7001**.

Struttura Dati Globale in RAM: Il Server deve avere un array globale per ricordarsi chi è connesso: ***int drone_tcp_sockets[10]; // Indice = ID del Drone, Valore = File Descriptor del socket TCP*** (Inizializzato tutto a ***-1***).

Comportamento (Multi-Thread): Il Server dovrà avere due thread principali in ascolto continuo:

- **Thread TCP (Accettazione):** Fa un ***while(1)*** con la ***accept()***. Quando un Drone si connette, riceve un pacchetto iniziale di tipo `LOGIN` con l'ID del drone. Il server salva il `client_socket` appena creato nell'array `drone_tcp_sockets[ID_DRONE]`.

Fatto questo, il server *non* chiude la connessione, ma la tiene lì pronta.

- **Thread UDP (Telemetria):** Fa un `while(1)` con una `recvfrom()`. Riceve pacchetti di tipo `TELEMETRY` contenenti: `ID_DRONE`, `Pos_X`, `Pos_Y`, `Batteria`.
  - Stampa a video: `[TELEMETRIA] Drone <ID> in posizione X:Y con batteria al Z%`.
  - **L'EVENTO CRITICO (La vera sfida dell'esercizio):** Se il Thread UDP legge che la batteria del Drone 3 è scesa **sotto il 15%**, deve mandare un segnale di allarme al drone. Come fa? Va a leggere l'array `drone_tcp_sockets[3]`, prende il socket TCP di quel drone, e gli spara dentro (usando `send()`) un pacchetto TCP di tipo `EMERGENCY_LAND`.

2. Il Drone (CLIENT)

All'avvio da terminale gli passi il suo ID (es. `./drone 3`).

1. **Fase di Setup (TCP):** Crea un socket TCP, fa la `connect()` alla porta 7000 del server e manda un pacchetto `LOGIN` con il suo ID (3).
2. **Lo Sdoppiamento (Multi-Thread):**
   - **Thread A (Sensori - UDP):** Crea un socket UDP. Entra in un `while(1)`. Ogni secondo (`sleep(1)`):
     - Genera X e Y casuali.
     - Decrementa una variabile locale `batteria` (che parte da 100) di un numero casuale tra 1 e 5.
     - Crea un pacchetto `TELEMETRY` e lo spedisce con `sendto()` al Server sulla porta 7001.
     - Se la batteria arriva a 0, il thread muore.
   - **Thread B (Comandi - TCP):** Il thread principale (o uno dedicato) fa una `recv()` **bloccante** sul socket TCP, aspettando ordini dalla stazione di terra.

- Se riceve il pacchetto `EMERGENCY_LAND`, stampa a caratteri cubitali `[ALLARME] BATTERIA CRITICA: ATTERRAGGIO DI EMERGENZA AVVIATO!`, chiude i socket ed esce forzatamente dal programma con `exit(0)`.

Struttura Pacchetto Consigliata

```
typedef struct {
    int type;       // 1 = LOGIN (TCP), 2 = TELEMETRY (UDP),
3 = EMERGENCY_LAND (TCP)
    int drone_id;   // L'ID del drone (es. 0, 1, 2...)
    int pos_x;
    int pos_y;
    int battery;    // Da 100 a 0
} Packet;
```